Engineer 김여은
Portfolio

Introduce.
직접 만들고, 부딪히고, 개선하며
기술을 실제 서비스로 연결하는 개발자로 성장하고 있습니다.

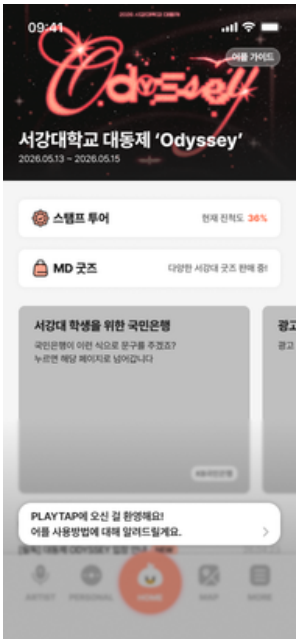
PlayTap

2025.12 ~ 진행중

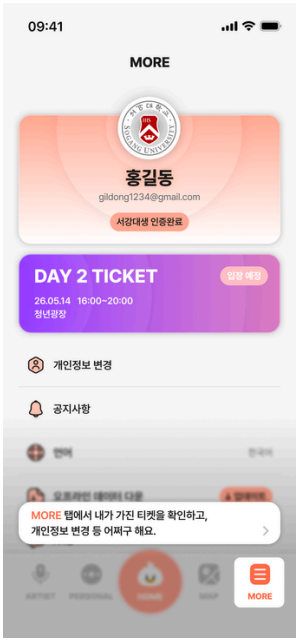
RFID 기반 축제 입장·권한·인원 관리 플랫폼

2026 서강대학교 축제 대상

Java, Spring Boot, JPA, PostgreSQL, Redis,
GraphQL, gRPC, REST API, WebSocket(STOMP),
AWS(ECS, RDS, S3), Flyway
github



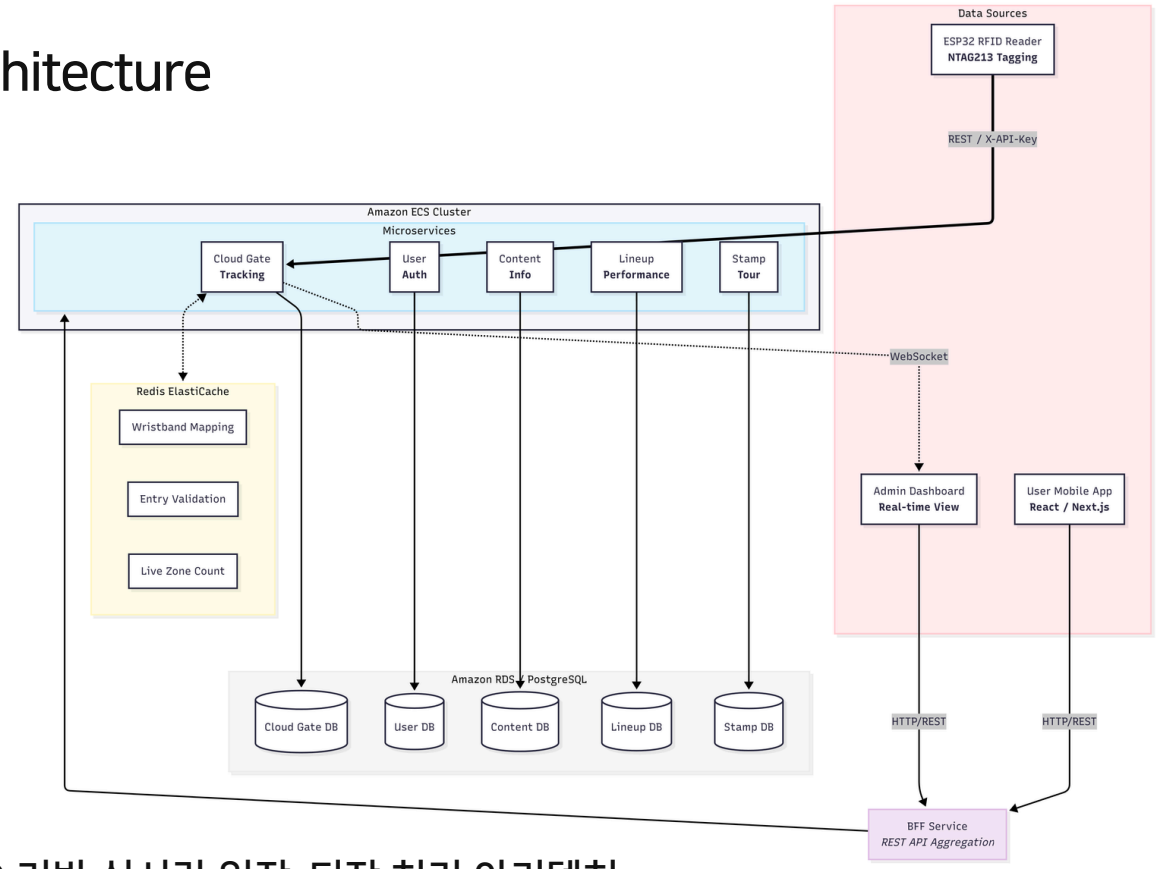




개요

PlayTap은 RFID 기반 입장 시스템과 클라우드 플랫폼을 결합해 대형 축제 현장의 운영 흐름을 디지털화하는 서비스입니다.
기존 종이·QR 티켓 운영에서 발생하던 암표 및 티켓 양도 문제, 대규모 입장 시 병목 현상, 구역별 권한 관리 혼선을 해결하기 위해 기획했습니다.
RFID 팔찌를 사용자 계정 및 티켓 정보와 1:1로 연동하고, 입장 판독·권한 검증·구역별 인원 집계를 실시간 처리할 수 있도록 설계했습니다.

Architecture



RFID 기반 실시간 입장·퇴장 처리 아키텍처

문제 인식

대형 축제 현장에서는 입장·퇴장 이벤트가 짧은 시간에 집중되기 때문에, 매 태그마다 DB를 직접 조회하면 응답 지연과 부하가 발생할 수 있습니다. 또한 운영자는 구역별 현재 인원 수를 실시간으로 확인할 수 있어야 합니다.

설계 판단

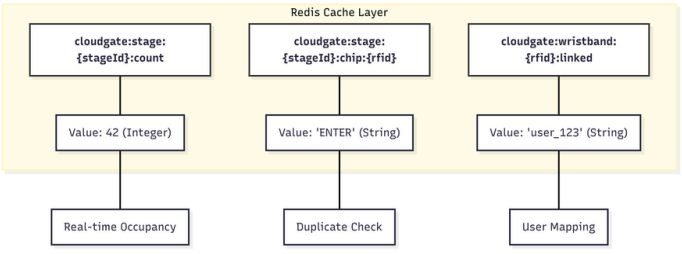
ESP32 리더기에서 RFID 태그를 인식하면 REST API를 통해 Cloud Gate Service로 이벤트를 전달하도록 설계했습니다. 서버는 태그 정보를 기반으로 리더기 상태와 방향(IN / OUT / RE_IN)을 확인하고, Redis 캐시를 활용해 팔찌 연동 정보와 칩 상태를 빠르게 검증합니다.

이를 통해 DB 조회를 최소화하고, 입장·퇴장 이벤트 처리 속도를 높였습니다.

Redis 기반 고속 판독 및 점유 상태 관리

Cloud Gate Service는 Redis를 활용해 팔찌 연동 정보, 칩의 마지막 상태, 구역별 현재 점유 수를 관리합니다.

- 매 판독 시 DB 조회 최소화
- 구역별 인원 수 실시간 집계
- 중복 입장 및 잘못된 퇴장 방지
- TTL 기반 일 단위 상태 만료 처리



실시간 점유율 브로드캐스트 구조

태그 이벤트 처리 후 TagEvent를 PostgreSQL에 저장하고, Redis 점유 카운터를 갱신합니다. 이후 WebSocket(STOMP)을 통해 클라이언트가 구독 중인 /topic/occupancy/{stageId} 경로로 구역별 점유 정보를 실시간 브로드캐스트합니다.
이를 통해 현장 운영자는 태그 직후 변경된 인원 수와 혼잡도를 거의 즉시 확인할 수 있습니다.

BE 업무

GraphQL · gRPC 기반 BFF 구조 설계

프론트엔드는 GraphQL 기반으로 화면 단위 데이터를 요청하고, BFF에서 요청을 해석해 내부 User / Content / Lineup / Stamp-tour / Cloud Gate Service를 gRPC 기반으로 호출하도록 설계했습니다.

- REST 대비 over/under-fetching 문제 감소
- 화면 응답 조합 책임을 BFF에 집중
- 프론트엔드 의존성 최소화
- 내부 서비스 간 통신 성능 개선
- 서비스별 책임 경계 명확화

ECS 기반 마이크로서비스 구조 설계

각 서비스를 독립 서비스로 설계 및 개발하고 Amazon ECS 환경에 분리 배포했습니다.
또한 서비스별 RDS를 분리하여 데이터 책임을 독립적으로 관리하도록 구성했습니다.

- 서비스 간 결합도 감소
- 장애 영향 범위 최소화
- 독립 배포 및 확장 가능
- 운영 안정성 향상
- 향후 도메인 확장 대응

세션 안정성 중심 토큰 전략

Access Token 만료 시 Refresh Token으로 세션을 재정립하고, Refresh Token 재발급 시 기존 토큰을 무효화하는 Rotation 전략을 적용했습니다.

- 세션 탈취 위험 감소
- Refresh Token 재사용 방지
- 안정적인 로그인 유지
- 보안성 강화

JoyCrew 2025.09 ~ 2025.12

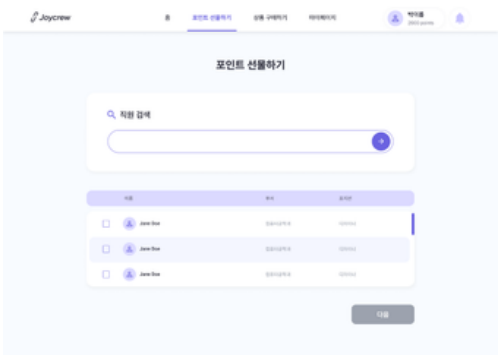
즉각적인 인정과 보상을 연결하는 B2B 복지 SaaS

기업 베타 테스트 이후 화이트라벨링 구조, EKS 무중단 배포, 모니터링 환경을 도입해 실서비스 운영 안정성 검증

Java, Spring Boot, JPA, AWS(EKS, RDS), 멀티 테넌트(화이트라벨링) 구조

github





개요

JoyCrew는 직원 간 즉각적인 인정과 보상을 연결하는 B2B 복지 SaaS 플랫폼입니다.
기업 베타 테스트 과정에서 트래픽 급증, 동시 요청, 데이터 분리 등 실제 운영 이슈를 직접 경험하며 구조를 개선했습니다.

핵심 문제 해결

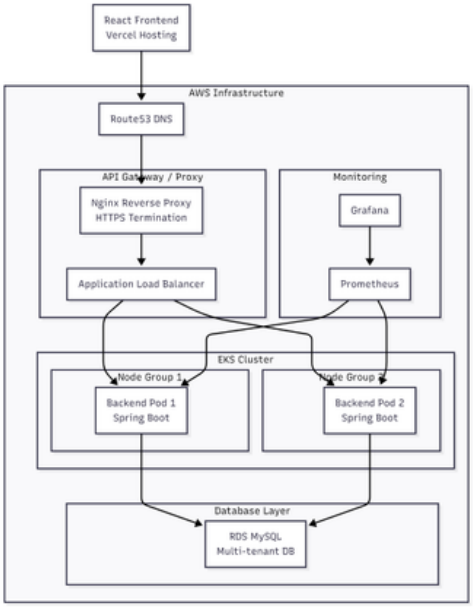
트랜잭션 안정성 중심 설계

문제

- 포인트 전송·리워드 교환 시 중복 요청으로 인한
- 포인트 이중 차감 및 잔액 불일치 발생

해결

- 포인트를 '자산 데이터'로 정의
- Spring Data JPA 비관적 락(PESSIMISTIC_WRITE) 적용
- 지갑(Row) 단위 배타적 잠금으로 Race Condition 원천 차단
- JMeter 기반 동시 요청 테스트(초당 100건)로 안정성 검증

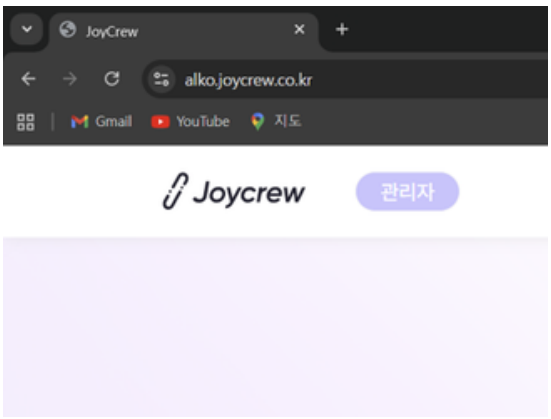


인프라 전환과 운영 안정성 확보

- 단일 EC2 병목 및 배포 중단 문제 확인
- AWS EKS 기반 인프라 전면 전환
- GitHub Actions → ECR → EKS CI/CD 파이프라인 구축
- Pod Replication, HPA 기반 자동 스케일링
- Rolling Update 기반 무중단 배포 구현

멀티 테넌트 · 화이트라벨링 구조 설계

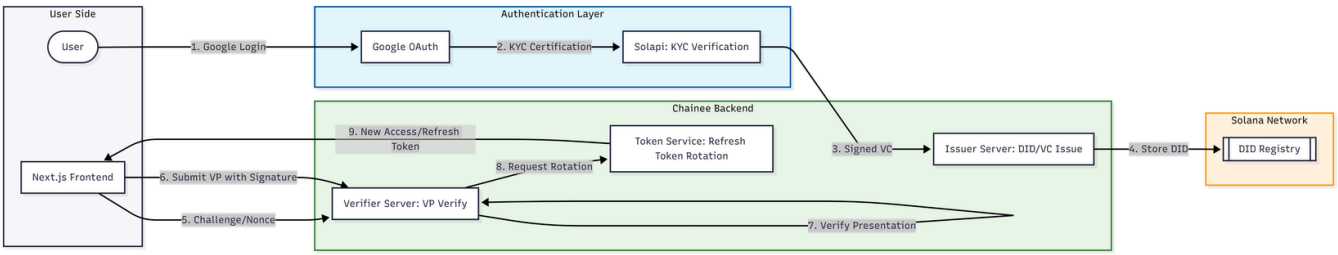
- 기업별 서브도메인 기반 Tenant 식별
- 모든 주요 엔티티에 tenant_id 중심 데이터 모델링
- 필터 레벨에서 테넌트 조건 자동 주입
- 단일 코드·단일 DB 환경에서도 기업별 데이터 완전 분리



Additional Engineering Challenges

Chainee

DID 기반 Web3 협업 플랫폼



핵심 초점:

Web3 요소를 신뢰 증명 수단으로 한정하고, 서비스 안정성은 Web2 백엔드에서 책임지는 구조 설계

주요 설계

- **OAuth** 로그인 이후 **KYC 인증** 선행 제어
- 인증 완료 시점에만 계정 활성화 → 미검증 사용자 차단
- 온체인 **상태 변화**를 서비스 도메인 상태로 변환·관리
- 계약 단계별 **상태 기반 알림** 생성
- 블록체인 지연·실패가 사용자 흐름을 차단하지 않도록 **책임 분리**

Presentalk

카카오톡 대화 기반 선물 추천 서비스

안정적인 상품 데이터 파이프라인 구축

공식 API가 제공되지 않는 환경에서 데이터 수집 제약을 극복하고, 안정적인 응답 성능을 확보

- 외부 커머스 플랫폼의 카테고리별 인기 상품을 스케줄러 기반으로 정기 수집하는 데이터 파이프라인을 구축
- 수집된 상품 데이터를 DB에 사전 적재하여, 외부 플랫폼 장애와 무관하게 빠르고 안정적인 API 응답을 제공하도록 설계